Assignment 1: Data Collection, Validation, Labelling and Versioning

1. Project Objective

An ML system is only as good as the data behind it. In this assignment your group builds the dataset for its course project and checks its quality with the tools covered in the course. You will collect data so that every sample can be traced back to its source, validate it automatically, label it with two or more annotators and measure how well they agree, extend it with augmentation and active learning, and version it with clean splits. What you freeze at the end becomes the fixed input to Assignment 2, so it is worth getting right.

Core Technical Pipeline:

- **Part A: Data Collection & Instrumentation**
  Building a pipeline that records the source, time, collector, tool version and hash of every sample.
- **Parts B and C: Validation, Labelling & Annotator Agreement**
  Catching data problems with automated checks, labelling with written guidelines, and measuring agreement.
- **Part D: Augmentation & Active Learning**
  Adding label-preserving augmentations and testing active learning against random selection.
- **Parts E and F: Versioning, Splits & Reproducibility**
  Versioning with DVC, making leakage-free splits, and recording what it takes to rebuild the data and labels.

Deadline

11:59 PM, 18 October 2026

Instructor: Dr. Yogesh K. Meena

2 October 2026 • Semester I (2026-2027) • CS 203 Software Tools and Techniques in AI (Course Project/Assignment 1)

© 2026 Indian Institute of Technology, Gandhinagar. All rights reserved.

1. Assignment Overview

The assignment has **six parts, A to F**, and it is the same for Projects 1, 4 and 5. Every group does the same parts for the same marks. Sections 2 to 7 describe each part in general terms. The details that differ between projects, such as minimum sizes, required checks and how to split the data, come from your own proposals and are collected in one place, Section 8. Read each part together with your project's entry there.

Learning Outcomes

By the end of this assignment you should be able to:

- instrument a data source so that every sample is traceable (source, time, tool, hash);
- write automated validation checks and use them to find and fix real data problems;
- design annotation guidelines, label data in Label Studio, and measure annotator agreement (Cohen's or Fleiss' kappa, WER between annotators);
- apply data augmentation and show that it keeps labels intact;
- compare active-learning selection with random selection under a fixed labelling budget;
- version data with DVC and create leakage-free splits, including a shifted split for later drift experiments;
- record the software versions and labelling process someone else would need to reproduce your data and labels.

Parts and Marks

Part	Title	Marks
A	Collection and instrumentation	15
B	Data validation	20
C	Labelling and agreement	25
D	Augmentation and active learning	20
E	Versioning and splits	10
F	Report and reproducibility	10
	Total	**100**

Section 9 explains how to submit, and Section 10 explains how submissions are checked.

2. Part A: Collection and Instrumentation

Start with the pipeline that brings data in. For every sample you should be able to say where it came from, when, who collected it and with which tool.

- **Collection pipeline:** Write a script or notebook that runs end to end and produces `metadata.csv` (or JSONL) with one row per sample. Each row records at least the source, timestamp, collector, tool version and a content hash.

- **Consent and licensing:** Write a short note on who provided the data, on what terms, and how you anonymised personal information. If you used outside material such as handbooks, web pages or public datasets, note its licence or terms of use. Keep an anonymised consent register, and do not submit signed forms or contact details.
- **Data flow:** Draw a diagram of how data moves from the source to `metadata.csv`, and keep the collection log.

3. Part B: Data Validation

Next, write checks that run automatically and catch problems in the raw data. Then fix what they find.

- **Validation suite:** Use Pandera, Great Expectations or pytest, with **at least 10 checks**. Between them they should cover schema and types, value ranges, missing values, duplicates, near-duplicates, label validity and train/test leakage. The checks listed for your project in Section 8 count towards the 10.
- **Before and after:** Run the suite on the raw data and report what it found in a before-and-after table: the checks you ran, the failures, and how you fixed each one. A suite that finds nothing is not believable.
- **Corrupted data:** Feed the suite deliberately corrupted data and show that it fails.

4. Part C: Labelling and Agreement

Here you design the labelling task, label with at least two people working on their own, and measure how much they agree.

- **Guidelines:** Write 1 to 2 pages of guidelines with definitions, edge cases and worked examples. Keep two versions: v1 for the first round, and v2 revised after you have looked at the disagreements.
- **Independent annotation:** Label in Label Studio with **at least 2 annotators** who work independently on an overlapping sample. The minimum sample size for your project is in Section 8.
- **Label order:** If an item gets more than one label, fix the order of the label levels before labelling starts and have every annotator follow it. Write the order down in your labelling protocol (Part F).
- **Agreement:** Report agreement before and after the guideline revision, using Cohen's or Fleiss' kappa for categories, or WER between annotators for transcripts. Add a confusion matrix and go through **at least 10 disagreements** in detail.
- **Gold test set:** Freeze a gold test set. It must not be used for tuning in any later assignment.

5. Part D: Augmentation and Active Learning

Grow the dataset with augmentations that keep labels intact, and test whether choosing what to label beats picking at random.

- **Augmentation:** Implement **at least 3 augmentations**. For each one, show on a sample that the labels still hold, using either a rule-based check or a human spot check.

- **Active learning:** Start from an unlabelled pool and pick items to label using an informativeness score. Compare this with random selection at the same budget, for example at 3 budget sizes. Plot a learning curve or label-efficiency curve and explain in a few sentences what it shows.

6. Part E: Versioning and Splits

Now version the data and cut the splits that Assignment 2 will use.

- **Versioning:** Version raw data, processed data and labels with DVC. A local folder or Google Drive is fine as the remote.
- **Splits:** Make train, validation and test splits that do not leak, split by the unit given for your project in Section 8, and add a shifted split for the drift experiments in Assignment 2.
- **Final tag:** Tag the repository `a1-final`. Assignment 2 starts from exactly this tag.

7. Part F: Report and Reproducibility

Finally, document the dataset and make sure someone outside your group could rebuild both the data and the labels. Rebuilding the data takes your code plus the exact software it ran on. Rebuilding the labels takes a record of how labelling was actually done, because the order in which annotators work can change the labels they give.

- **Dataset card:** sources, size, class balance and known biases.
- **Clean repository:** no leftover or temporary files, and no hard-coded local paths.
- **README:** reproduces the data in **at most 5 commands**.
- **Data reproducibility:** the software versions, models and seeds described below.
- **Label reproducibility:** a labelling protocol, described below.

Data Reproducibility

List every piece of software your data depends on, with its exact version.

- **Python packages:** Pin every package in `requirements.txt` (`pandas==2.2.2`, not `pandas`). Also include the full `pip freeze` or `conda env export` output from the environment you actually used, as `requirements_freeze.txt`.
- **Other software:** In `ENVIRONMENT.md`, list everything that the package list does not cover, with versions: Python itself, Label Studio, the DVC command-line tool, any audio or model-serving tools, your operating system, and the GPU and CUDA version if you used one.
- **Models:** For every model that produced data or labels, give the name, the exact version, and where it came from, such as a Hugging Face revision hash or a model tag.
- **Randomness:** Fix the seeds used for sampling, augmentation, active-learning selection and splitting, and record them in `ENVIRONMENT.md`.

Label Reproducibility

Write a labelling protocol, `LABELLING_PROTOCOL.md`, that lets someone repeat your labelling the same way you did it. It should cover:

- **Who labelled what:** anonymised annotator IDs, and which items each annotator labelled in each round.
- **Tools and guidelines:** the Label Studio version and project config used in each round, and which guideline version (v1 or v2) each round followed.
- **Order of label levels:** the order in which the levels of a multi-level label were filled in, why you chose it, and confirmation that every annotator used the same order. Order matters here. An annotator who picks a category first may write the free-text part of the label, such as a transcript or an edit, differently from one who writes it first. Your project's label levels are listed in Section 8.
- **Order of items:** the order in which items were shown to annotators, with the seed if they were shuffled.
- **What annotators could see:** whether they saw model predictions, pre-annotations or each other's labels while working.
- **Disagreements:** how they were resolved, and who made the final call.
- **Timeline:** the date of each labelling round, matched to the Label Studio export files.

8. Project-Specific Requirements

Everything that differs between projects is collected here. Each entry adds to the general requirements in Sections 2 to 7; it does not replace them.

Project 1: AgentEvalOps

- **Collection (Part A):**
  - Build the typed tool layer (retrieval, calculator, structured data and file operations, `submit_answer`) and the function-calling agent loop in plain Python.
  - Build the trace logger with MLflow Tracing. Each trace records the task, configuration hash, model and prompt version, every LLM call and tool call with its arguments and outputs, errors, latency, token usage and the final answer.
  - Create the task suite: 5 or 6 templates expanded to 20 to 30 instances, with gold answers computed in code. Include a few ambiguous or unanswerable tasks.
- **Validation checks (Part B):** trace schema; types and arguments of tool calls; gold answers recomputed and compared; determinism of the tool layer; duplicate tasks; answers leaking into prompts.
- **Labelling (Part C):**
  - Run the agent to collect trajectories. A few hundred are expected.
  - Two annotators label 40 to 50 stratified trajectories. Each gets an outcome, a primary failure category (wrong tool, malformed arguments, hallucinated result, loop, premature stop, correct process with wrong answer, or no failure) and the step where the first error happens.

  - Report Cohen's kappa and revise the taxonomy. Then measure how well simple automatic detectors (for loops and malformed calls) match the human labels.

- **Augmentation and active learning (Part D):**
  - Augmentations: task variants with changed entities and parameters, fault injection (tool failure, slow response, changed output format), and prompt paraphrases.
  - Active learning: pick trajectories to label using signals such as step count, repeated calls and errors, compare with random picks, and report the failure coverage each approach gives.
- **Splits (Part E):** hold out whole template families for the test set, not just instances. The shifted split is the fault-injected tasks. Version tasks and traces with DVC.
- **Label levels (Part F):** outcome, primary failure category, and first-error step.

Project 4: Course Advisory SLM

- **Collection (Part A):**
  - Parse curriculum handbooks, credit-requirement sheets and course catalogues into structured JSON: course code, title, credits, prerequisites, core or elective type, and programme (CSE, EE and so on).
  - Log the source document and page for each record.
  - Build the prerequisite dependency graph and the deterministic rule engine that computes ground truth, and use the rule engine to generate the synthetic student transcripts.
- **Validation checks (Part B):** every prerequisite code exists, the prerequisite graph has no cycles, credit totals match the handbook, and no course code appears twice. Synthetic transcripts pass the rule engine, with no prerequisite violations and credit caps respected. No student or template appears in both train and test.
- **Labelling (Part C):**
  - Build at least 300 advisory query-response pairs covering remaining core requirements, eligibility, and elective recommendations with justification. Gold answers come from the rule engine.
  - Humans audit the LLM-paraphrased queries and justifications in Label Studio: accept, edit or reject, with an error category. Two annotators audit at least 60 items; report kappa.
  - Collect at least 30 real queries through Google Forms as an out-of-distribution set.
- **Augmentation and active learning (Part D):**
  - Augmentations: paraphrasing, typos and abbreviations, and varied transcript length and ordering. The rule engine confirms that the answer has not changed.
  - Active learning: give priority to examples where a base model's answer disagrees with the rule engine, and compare with random picks.
- **Splits (Part E):** split by student and by programme, not by row. The shifted split is the real Google Forms queries. Version the curriculum JSON and the datasets with DVC.
- **Label levels (Part F):** the accept, edit or reject decision, any edit made, and the error category.

Project 5: Voice Helpdesk ASR

- **Collection (Part A):**
  - Record at least 2 hours of speech from at least 15 speakers (30 or more for the final project), with their consent. Include scripted and spontaneous helpdesk queries (admissions, hostel and mess, IT, student services) that mix English, Hindi and Gujarati.
  - Record metadata for each clip: speaker ID, region, device, noise condition and language mix. Log the sample rate, duration and recording tool automatically.
- **Validation checks (Part B):** sample rate, channels, duration bounds, clipping, silence ratio, estimated SNR, duplicate clips (by audio hash), characters outside the agreed script, plausible characters per second, and a language-ID check on each clip.
- **Labelling (Part C):**
  - Transcribe and intent-label in Label Studio, following written rules for code-mixing, script and normalisation.
  - Double-annotate at least 100 clips. Report WER between annotators for the transcripts, and Cohen's or Fleiss' kappa for intent.
- **Augmentation and active learning (Part D):**
  - Augmentations: MUSAN and room-impulse-response noise and reverberation, speed and pitch perturbation, and codec compression. Report off-the-shelf Whisper WER on clean and on augmented clips.
  - Active learning: choose clips to transcribe by low decoder confidence, and compare with random picks.
- **Splits (Part E):** keep speakers disjoint across train, validation and test. The shifted split is Cohort B, a held-out accent or region group. Version audio, transcripts and metadata with DVC.
- **Label levels (Part F):** the transcript and the intent label.

9. Submission

Submit by **11:59 PM, 18 October 2026**. There is no earlier checkpoint.

- Put everything in one zip file named `CS203_A1_Project<N>_<roll-numbers>.zip` (for example, `CS203_A1_Project4_25210087_25210122.zip`).
- Upload it to Google Drive or OneDrive and set sharing so that anyone with the link can download it.
- Open the Assignment 1 post on Google Classroom, attach the link, and turn it in. Do not change the file after the deadline.

The zip holds four folders:

- `code/`: the repository at tag `a1-final`, including its `.git` folder, with the GitHub URL in the README. It contains:
  - the code and notebooks for every part, the DVC files, guidelines v1 and v2, and the Label Studio config;

  - `README.md` (rebuilds the data in at most 5 commands), `requirements.txt`, `requirements_freeze.txt`, `ENVIRONMENT.md`, `LABELLING_PROTOCOL.md`, and `HANDOFF_A2.md` (frozen files, split definitions, label schema and gold-set checksum);
  - `prompts/`: every prompt used to automate any part of the work with an AI model, exactly as used. That means the prompts your code sends to a model, with the model and version, and the prompts you gave AI assistants such as ChatGPT, Claude or Copilot, with the file the output went into.
- `data/`: the frozen dataset at `a1-final` (raw and processed data, labels, gold test set, split manifests and `metadata.csv`). It must match what DVC versions at `a1-final`, with personal information anonymised.
- `supporting_files/`: everything that backs a number or finding in the report, organised however you like. This includes the raw Label Studio exports from each annotator, the agreement calculations, validation outputs on raw, fixed and corrupted data, logs, augmentation samples, the data behind each plot, and a 2-minute screen recording of the pipeline running end to end.
- `report/`: `report.pdf`, at most 6 pages plus appendices. It covers:
  - Parts A to F: method, validation findings, agreement results, augmentation and active-learning results, splits, the dataset card, and a summary of software versions and the labelling protocol;
  - a contribution table showing who did what;
  - an appendix that indexes the submission, giving for each number and finding the file in the zip that supports it;
  - an appendix describing how you used AI assistance.

Before you submit, unzip the file into a new folder and check that your README commands rebuild `data/`.

10. Evaluation and Verification

Marks follow the table in Section 1. Full marks need evidence (logs, screenshots, commit history, numbers), not just a description of what you did.

How Your Submission Will Be Checked

Every submission is cross-checked by both **AI** and the **TAs**, each looking at different things.

- **AI checks implementation validity.** It checks that your code actually covers the data you submitted (every file in `data/` is collected, validated, labelled and split by code in `code/`), that the prompts in `code/prompts/` are valid and are the ones your code really uses, and that every number and finding in the report matches the file it points to.
- **TAs check the reported figures and coverage.** They check the numbers in the report against the supporting files, and go through the assignment part by part to see how much of it the submission covers.

Final evaluation and marking take the AI findings into consideration. In practice this means:

- Every number in the report should point to the file behind it through the index appendix. A number with nothing behind it is treated as unsupported.

- Submit prompts and exported files exactly as they were. If you had to correct an export, include the original, the corrected file and the script that made the change.
- Use relative paths and pinned versions so the code runs straight from the unzipped folder.

Individual Contribution

We will look at your project's git history to assess each member's contribution. This only affects individual marks where contributions differ a lot between members. Commit your own work from your own account so that the history shows who did what.

Important Notes

- Each member must be able to explain any code or number in the submission.
- The Institute Honour Code applies: `https://iitgn.ac.in/students/honourcode`

© 2026 Indian Institute of Technology, Gandhinagar. All rights reserved.